CENTRO UNIVERSITÁRIO FIAP – PAULISTA

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS – 1TDSPH

CARLOS HENRIQUE FRANCO DE MELO — RM569868

HENRIQUE BONACHELA DE CARVALHO CARABANTE — RM573620

MURILO ALMEIDA RODRIGUES DE SOUZA — RM573977

SOLUÇÃO: **ECOSCORE**

EQUIPE: **SOLCON**

SÃO PAULO - SP

2026

Sumário

1 OBJETIVO....3

2 SISTEMA E FUNCIONALIDADES....3

3 TABELA DE ENDPOINTS....4

3.1 Endpoint Ação....4

3.2 Endpoint Missão....5

3.3 Endpoint Post....5

3.4 Endpoint Quiz....6

3.5 Endpoint Ranking....6

3.6 Endpoint Usuário....7

4 PROTÓTIPOS DO SISTEMA....8

4 MODELO DE ENTIDADE-RELACIONAMENTO....12

5 DIAGRAMA DE CLASSES....13

5.2 Diagrama de Classes (DAO)....14

5.3 Diagrama de Classes (BO)....15

5.4 Diagrama de Classes (Resource)....16

6 PROCEDIMENTOS PARA RODAR A APLICAÇÃO....17

6.1 Requisitos....17

6.2 Execução....17

1 OBJETIVO

Nosso objetivo é desenvolver um sistema de gamificação que incentive usuários da SoulUp a realizarem ações reais em troca de recompensas e pontos, mostrando descrições do progresso de impacto que ele causou. Essas ações geram posts que permitem interação da

Nosso propósito é mostrar como seriam calculadas as ações sustentáveis de forma justa e como as diferentes métricas mudam a pontuação, dando um exemplo de como seria um perfil de usuário internamente.

2 SISTEMA E FUNCIONALIDADES

Nossa API feita com Java + Spring Boot acessa um usuário do banco de dados, que tem atributos: nome, id, confiabilidade, saldo de pontos, selos conquistados e atividade recente, além de informações de missões, ações, posts e quizzes, todos com CRUD completo. Além disso, a solução tem um interfáce gráfica simples para testes, que são os pacotes *view, e controller* da última sprint entregue.

A solução tem aas seguintes classes:

- Acao – Classe que pode ter diferentes tipos que impactará no cálculo dos pontos gerados por ela, que são calculados pela classe *CalculadoraPontos*.
- Quiz – Classe que gerá pontos dependendo de números de acertos e quantidade de pontos por acerto
- Missao – Classe que gerá pontos e selos para o usuário
- Post – Classe que adiciona votos ao ranking daquele usuário
- Ranking – Um ranking semi-funcional que muda conforme os número de votos recebidos. Cada objeto/registro é uma posição de um usuário no ranking.

3 TABELA DE ENDPOINTS

As tabela a seguir são para cada uma das classes, mas há um arquivo markdown em “/docs/endpoints-table.md” na pasta do projeto que contém no topo uma tabela geral além de tabelas divididas por classe. E também há um arquivo yaml em “/docs/endpoint-insomnia.yaml” com todas essas requisições prontas para serem importadas pro Insomnia.

3.1 Endpoint Ação

Nome	Método	Endereço	Status
findAllAcao	GET	http://ecoscore-java-db.onrender.com/acao	200, 500
findByIdAcao	GET	http://ecoscore-java-db.onrender.com/acao/:id	200, 404, 500
saveAcao	POST	http://ecoscore-java-db.onrender.com/acao	201, 400, 500
updateAcao	PUT	http://ecoscore-java-db.onrender.com/acao/:id	200, 400, 404, 500
deleteAcao	DELETE	http://ecoscore-java-db.onrender.com/acao/:id	204, 404, 500

3.2 Endpoint Missão

Nome	Método	Endereço	Status
findAllMissao	GET	http://ecoscore-java-db.onrender.com/missao	200, 500
findByIdMissao	GET	http://ecoscore-java-db.onrender.com/missao/:id	200, 404, 500
saveMissao	POST	http://ecoscore-java-db.onrender.com/missao	201, 400, 500
updateMissao	PUT	http://ecoscore-java-db.onrender.com/missao/:id	200, 400, 404, 500
deleteMissao	DELETE	http://ecoscore-java-db.onrender.com/missao/:id	204, 404, 500

3.3 Endpoint Post

Nome	Método	Endereço	Status
findAllPost	GET	http://ecoscore-java-db.onrender.com/post	200, 500
findByIdPost	GET	http://ecoscore-java-db.onrender.com/post/:id	200, 404, 500
savePost	POST	http://ecoscore-java-db.onrender.com/post	201, 400, 500
updatePost	PUT	http://ecoscore-java-db.onrender.com/post/:id	200, 400, 404, 500
deletePost	DELETE	http://ecoscore-java-db.onrender.com/post/:id	204, 404, 500

3.4 Endpoint Quiz

Nome	Método	Endereço	Status
findAllQuiz	GET	http://ecoscore-java-db.onrender.com/quiz	200, 500
findByIdQuiz	GET	http://ecoscore-java-db.onrender.com/quiz/:id	200, 404, 500
saveQuiz	POST	http://ecoscore-java-db.onrender.com/quiz	201, 400, 500
updateQuiz	PUT	http://ecoscore-java-db.onrender.com/quiz/:id	200, 400, 404, 500
deleteQuiz	DELETE	http://ecoscore-java-db.onrender.com/quiz/:id	204, 404, 500

3.5 Endpoint Ranking

Nome	Método	Endereço	Status
findAllRanking	GET	http://ecoscore-java-db.onrender.com/ranking	200, 500
findByIdRanking	GET	http://ecoscore-java-db.onrender.com/ranking/:id	200, 404, 500
saveRanking	POST	http://ecoscore-java-db.onrender.com/ranking	201, 400, 500
updateRanking	PUT	http://ecoscore-java-db.onrender.com/ranking/:id	200, 400, 404, 500
deleteRanking	DELETE	http://ecoscore-java-db.onrender.com/ranking/:id	204, 404, 500

3.6 Endpoint Usuário

Nome	Método	Endereço	Status
findAllUsuario	GET	http://ecoscore-java-db.onrender.com/usuario	200, 500
findByIdUsuario	GET	http://ecoscore-java-db.onrender.com/usuario/:id	200, 404, 500
saveUsuario	POST	http://ecoscore-java-db.onrender.com/usuario	201, 400, 500
updateUsuario	PUT	http://ecoscore-java-db.onrender.com/usuario/:id	200, 400, 404, 500
deleteUsuario	DELETE	http://ecoscore-java-db.onrender.com/usuario/:id	204, 404, 500

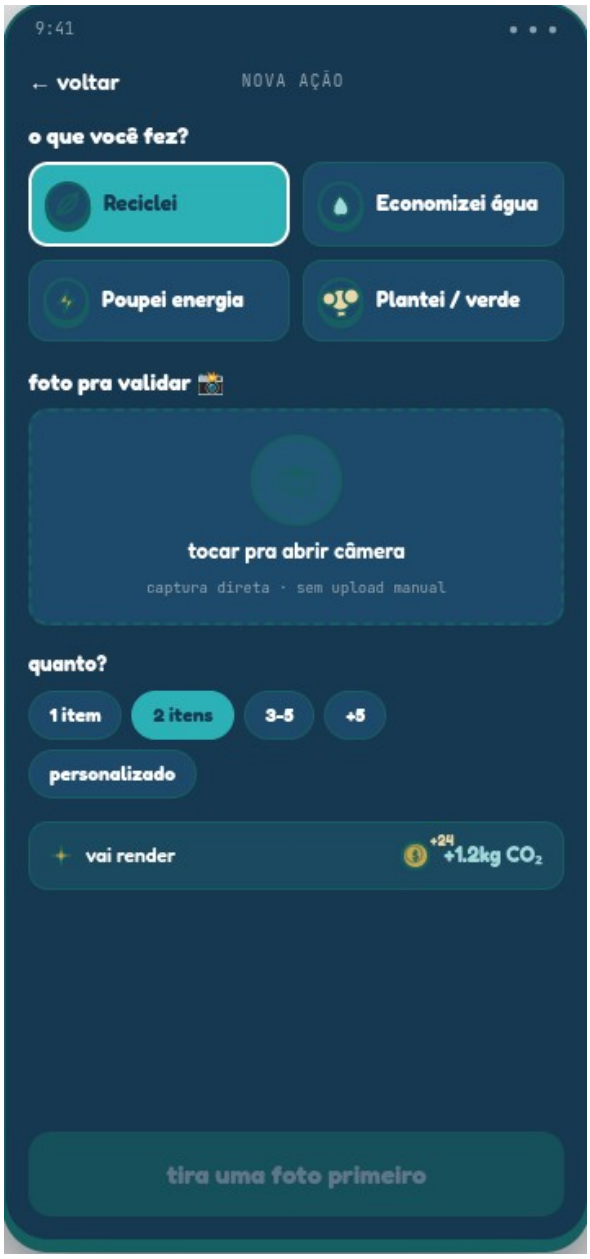
4 PROTÓTIPOS DO SISTEMA

Imagem 1 – Protótipo



A tela acima, é a entrada do dashboard, o botão “voltar a brilhar” muda conforme a última vez que o usuário realizou algo, ela permite **Registrar uma Ação.** Que é uma requisição GET na API, no caminho “/usuario”.

Imagem 2 - Protótipo



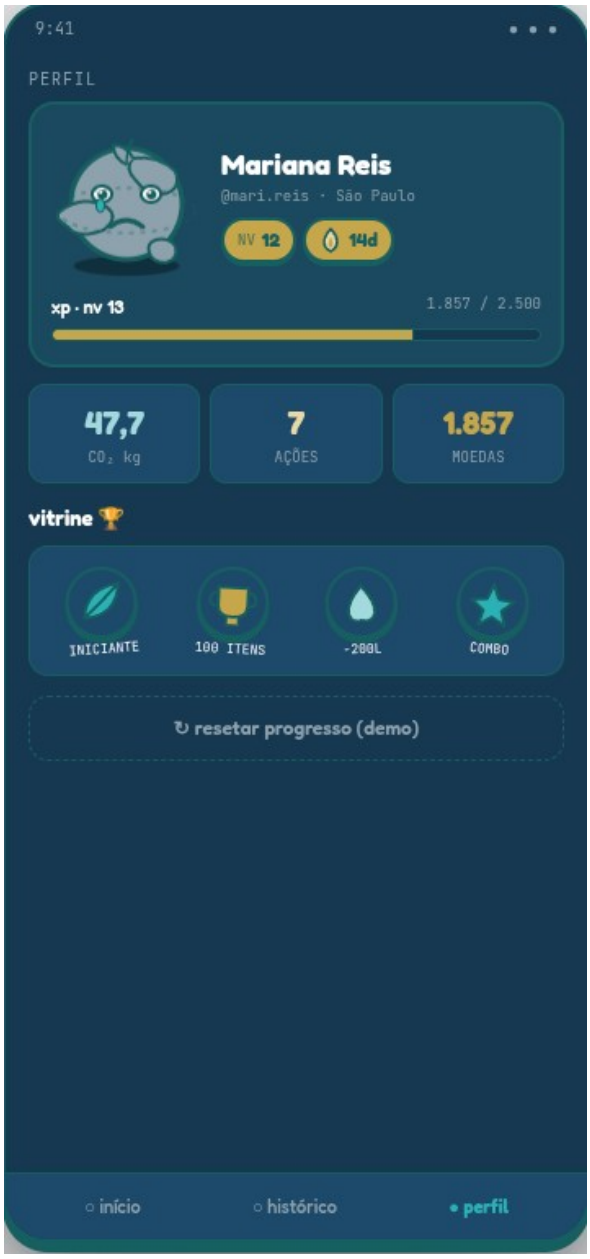
Ao clicar abre a tela de registro que permite mudar o tipo de ação, enviar uma validação dela e registrar detalhes para serem calculados pelo sistema.

Imagem 3 - Protótipo



A partir da tela principal é possível clicar em histórico para ver estatísticas e **atividades recentes**. Isso usa a requisição de “/usuario” para pegar o campo “*atividadeRecente*”.

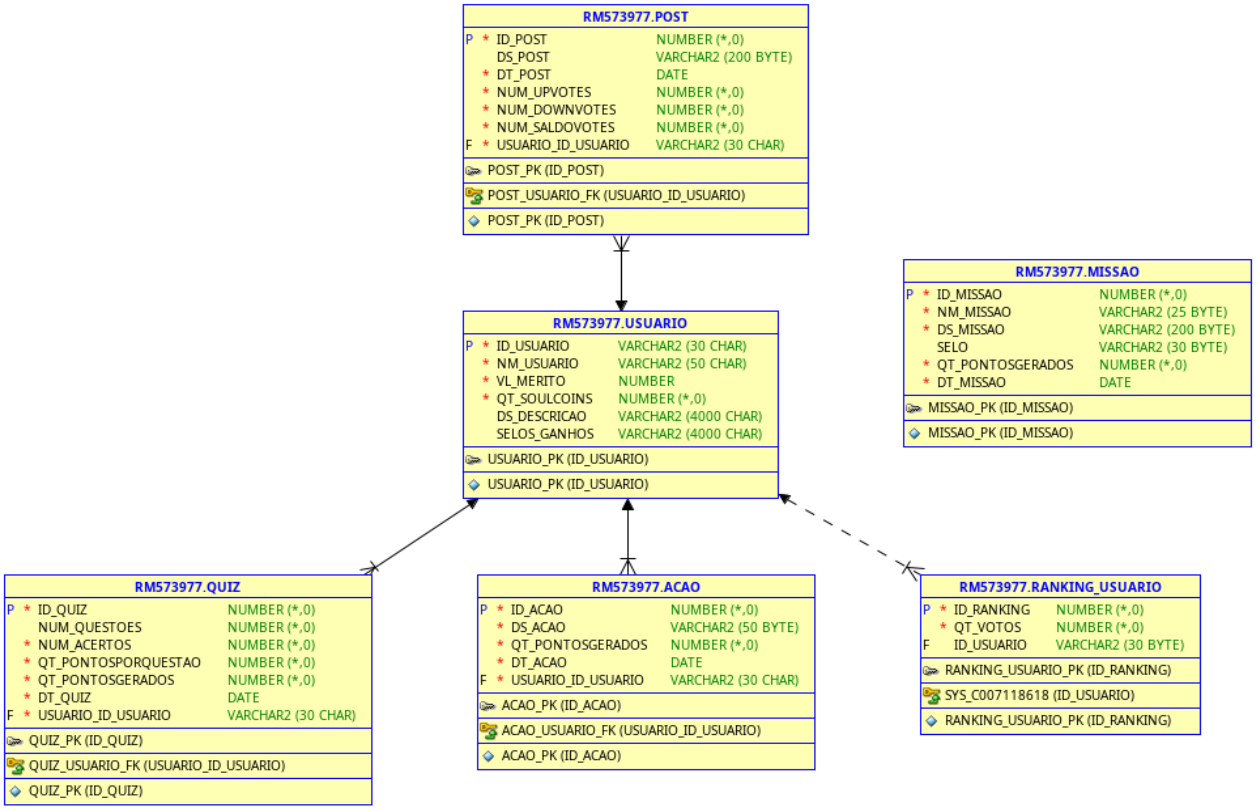
Imagem 4 - Protótipo



Também é possível ver o perfil e personalizar exibição de selos. Que também usa a requisição de “/usuario”

4 MODELO DE ENTIDADE-RELACIONAMENTO

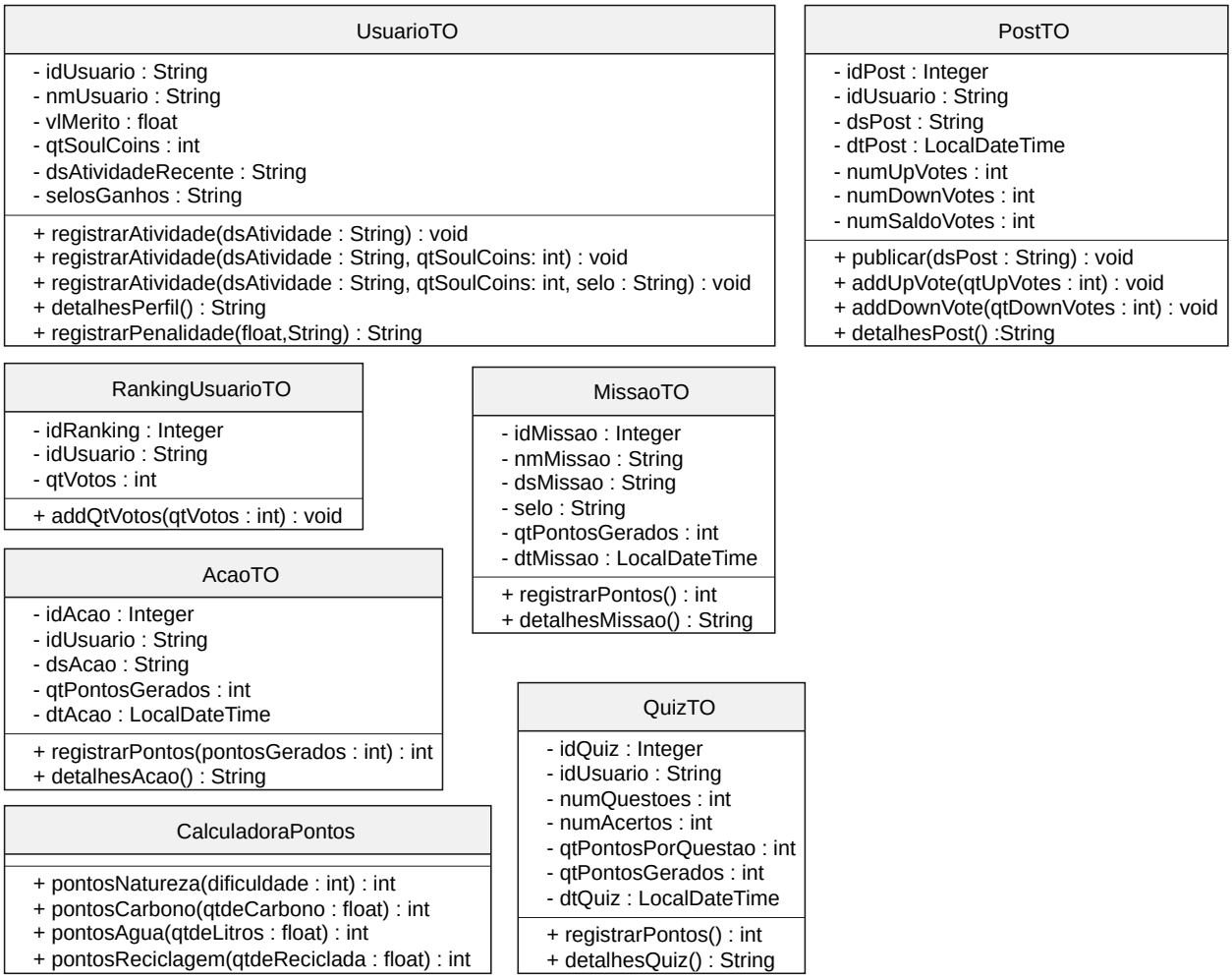
Imagem 5 – MER



Fonte: Desenvolvido pela Solcon (2026).

5 DIAGRAMA DE CLASSES

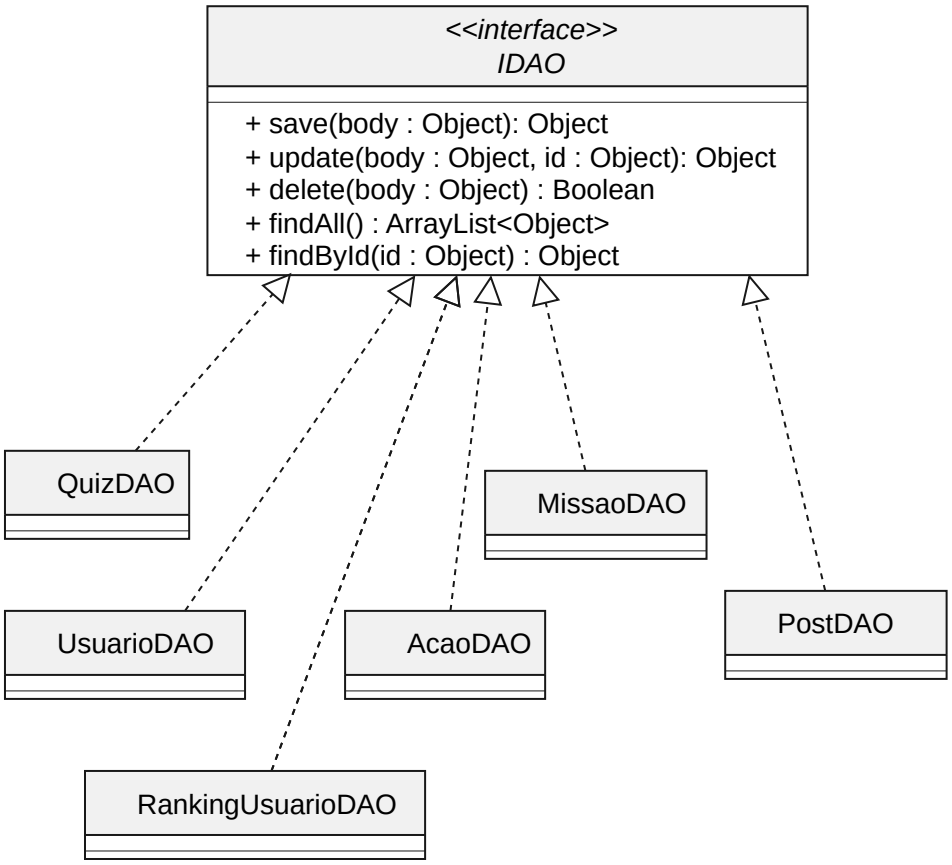
Imagem 6 – Diagrama de classes



Fonte: Desenvolvido pela Solcon (2026).

5.2 Diagrama de Classes (DAO)

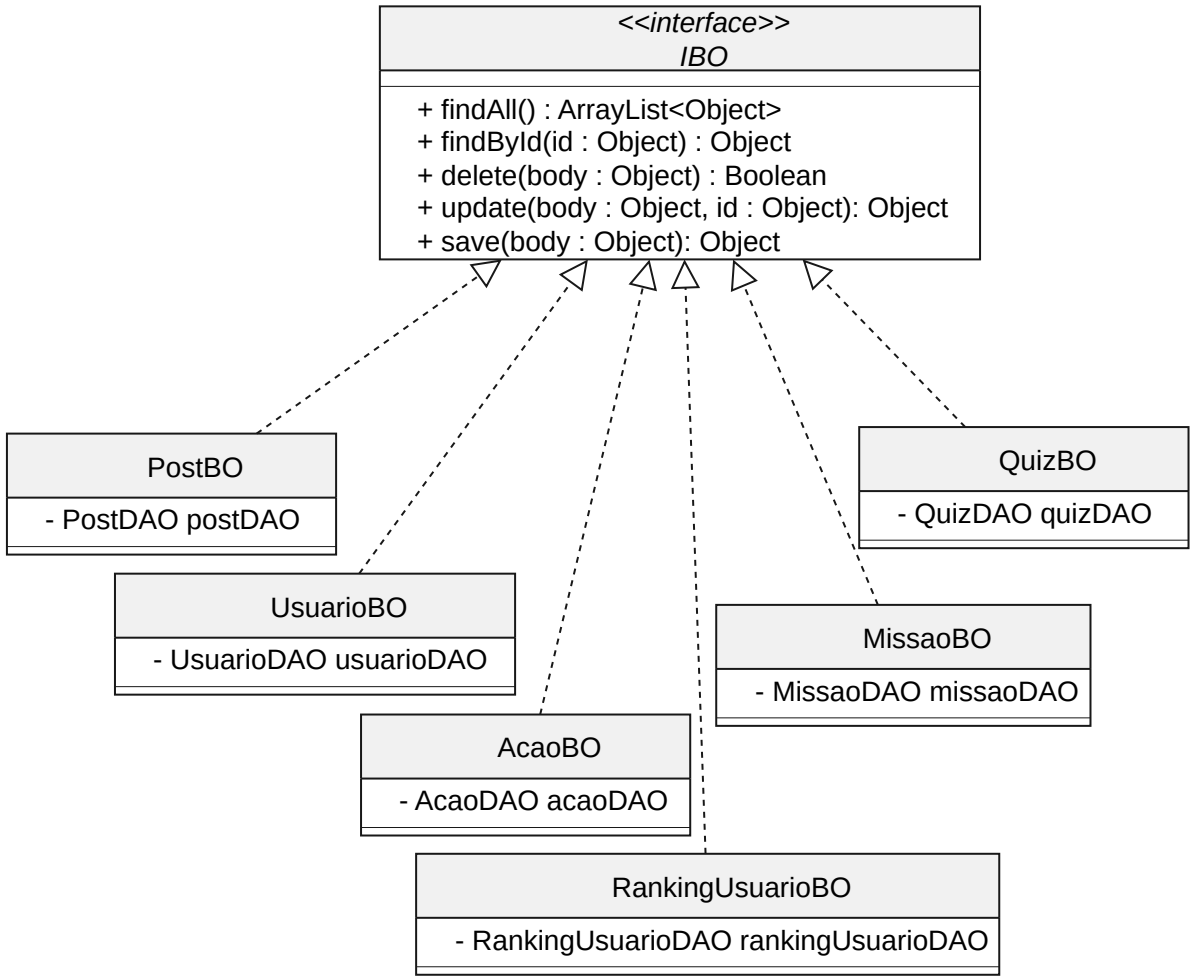
Imagem 7 – Diagrama de classes DAO



Fonte: Desenvolvido pela Solcon (2026).

5.3 Diagrama de Classes (BO)

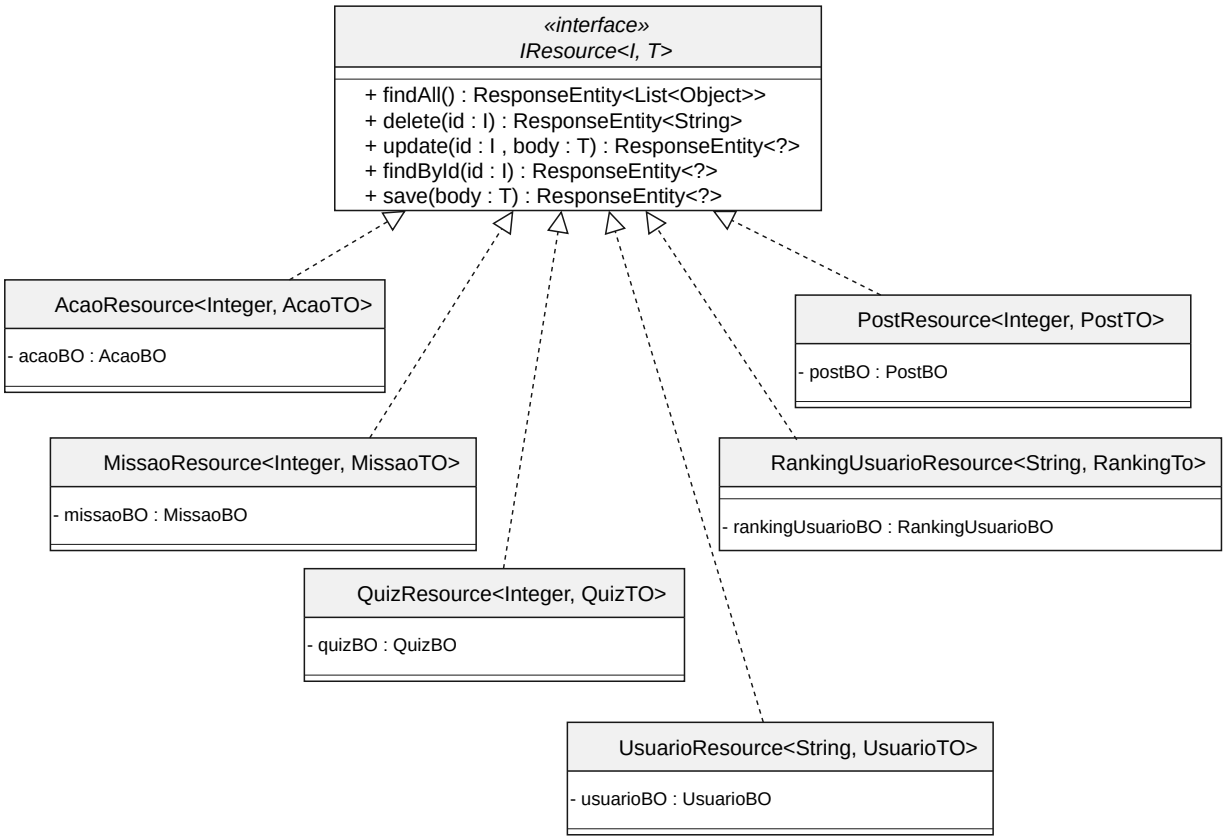
Imagem 8 – Diagrama de classes BO



Fonte: Desenvolvido pela Solcon (2026).

5.4 Diagrama de Classes (Resource)

Imagem 7 – Diagrama de classes Resource



Fonte: Desenvolvido pela Solcon (2026).

6 PROCEDIMENTOS PARA RODAR A APLICAÇÃO

A aplicação foi desenvolvida em **Java**, utilizando o **IntelliJ IDEA** como ambiente de desenvolvimento e o **JDK 21** para compilação e execução. O sistema utiliza **Oracle Database** para persistência dos dados, com modelagem realizada no **Oracle Data Modeler** e geração do script DDL, executado posteriormente no **Oracle SQL Developer**.

6.1 Requisitos

Para execução do sistema (API local ou interfáce gráfica), são necessários:

- IntelliJ IDEA;
- Java JDK 21;
- Oracle SQL Developer (por mais que o programa permitar ver dados do banco);

Para visualização da API, são necessários:

- Navegador para requisição GET da API ou um programa (como o Insmonia) que possa fazer requisições HTTP.

6.2 Execução

Se necessário baixe o projeto da última release disponível e entre na pasta Ecoscore.

Execução da interfáce de testes: O projeto deve ser aberto no IntelliJ IDEA, verificando-se a configuração do **JDK 21,** na pasta src/main procure a classe br.com.fiap.view.RunApplication.

Execução da API para localhost: O projeto deve ser aberto no IntelliJ IDEA, verificando-se a configuração do **JDK 21** funcionametno do Spring Boot**,** na pasta src/main procure a classe br.com.fiap.EcoscoreApplication.

Uso da API publicada: Abrir o navegador (para requisições GET) ou o Insomnia para demais requisições e faça a requisição com a URI do endpoint, considere que pode haver um cooldown de aticamento da API no Render.